

# Interpretable IoT Flow Classification: Random Forest Model with SHAP Analysis on RT-IoT2022

Archibald Emmanuel Carrion Claeys  
PPCI (ECCI/CITIC?)  
Universidad de Costa Rica  
San Jose, Costa Rica  
archibald.carrion@gmail.com

Dr. Adrián Lara  
PPCI

**Abstract**—As IoT devices become increasingly prevalent in modern households, their network attack surface expands proportionally, making intrusion detection a critical challenge. This work addresses the need for both accurate and interpretable IoT traffic classification by training a Random Forest model on the RT-IoT2022 dataset (UCI ID 942), which contains 81 flow-level features across 12 labeled attack and benign classes. Our model achieves 99.86% accuracy and a macro-averaged F1 score of 0.971 on the test set. To ensure interpretability, we apply SHAP analysis directly to the raw features, using class-balanced visualizations to reveal how individual features (e.g., destination ports, packet rates) drive predictions for specific attack types. The results demonstrate that interpretable models can achieve strong performance while providing actionable insights for security analysts.

**Index Terms**—IoT security, network flow classification, random forest, decision tree, SHAP, RT-IoT2022, intrusion detection

## I. INTRODUCTION

As IoT devices become more and more popular in modern households, so does their network attack surface. As recent news events illustrate, these devices remain highly vulnerable to network attacks and exploitation techniques. Understanding how IoT networks operate and how various attack strategies leave distinct signatures in network flow data is therefore a problem of growing practical importance.

The RT-IoT2022 dataset [1] (UCI ID 942), with its 81 flow-level features and 12 labeled attack and benign classes

### A. Supervised Multi-Class Classification

Intrusion detection in this context is framed as a *supervised multi-class classification* problem: given a labelled training set of flow-feature vectors  $\mathbf{x}_i \in \mathbb{R}^d$  with corresponding class labels  $y_i \in \{1, \dots, K\}$ , the goal is to learn a function  $f: \mathbb{R}^d \rightarrow \{1, \dots, K\}$  that generalises to unseen flows. Here  $d = 81$  and  $K = 20$  for RT-IoT2022.

### B. SHAP for Model Interpretability

SHAP (SHapley Additive exPlanations) [2] provides a theoretically grounded framework for explaining the output of any machine learning model. The SHAP value  $\phi_j$  for feature  $j$  and a given prediction is derived from Shapley values in cooperative game theory: it represents the average marginal contribution of feature  $j$  to the prediction across all possible subsets of features.

The SHAP values are used because:

- Directional: they show whether a feature pushes toward or away from a class
- Model-agnostic, can explain or at least explain an approximation of any model, even black box like neural networks

## II. PREVIOUS WORK

When researching the intersection of machine learning (ML) and Internet of Things (IoT) security, we found both broad surveys that map the landscape and narrower applied studies that benchmark specific classifiers on specific datasets. This section reviews the work most directly relevant to the present paper, organized into three groups: surveys, studies sharing the same datasets, and studies sharing the same methodology.

### A. Surveys

Two recent surveys frame the problem space well. Alfahaid et al. [3] conduct a comprehensive review of ML-driven IoT security solutions published between 2020 and 2024, covering supervised, unsupervised, and reinforcement learning approaches as well as advanced techniques such as federated and transfer learning.

Santhosh Kumar et al. [4] survey ML-based intrusion detection systems (IDS) for IoT communication, evaluating solutions against quality-of-service metrics and proposing a fuzzy convolutional neural network (CNN) classifier as a benchmark.

### B. Studies on RT-IoT2022

Four prior studies use the same primary dataset as this work.

Sharmila and Nagapadma [1], who also introduced the RT-IoT2022 dataset, propose a quantized autoencoder (QAE) for anomaly detection on resource-constrained edge devices. Their approach is unsupervised (the autoencoder is trained only on benign traffic and detects attacks via high reconstruction error) which means it cannot distinguish between attack *types*, only flag deviations from normal behaviour. No feature importance or explainability analysis is provided.

Airlangga [5] compares four supervised classifiers (random forest (RF), gradient boosting (GB), logistic regression (LR), and multi-layer perceptron (MLP)) on RT-IoT2022, reporting

that both RF and GB achieve perfect scores across all metrics. However, the study does not investigate which features drive those predictions, and no explainability method is applied.

Sama et al. [6] extend this comparison further, evaluating k-nearest neighbours (KNN), GB, XGBoost, support vector machine (SVM), RF, decision tree (DT), and extremely randomised trees (ERT) on the same dataset. Again, no explainability analysis accompanies these results.

Tuteja et al. [7] apply an RF classifier to RT-IoT2022, using the synthetic minority oversampling technique (SMOTE) to address class imbalance and reaching an overall accuracy of 98%. Their feature importance analysis highlights flow duration and protocol type as the strongest predictors of anomalous behaviour. While the reported per-class precision, recall, and F1 scores confirm that RF handles RT-IoT2022 well, the study does not apply SHAP and do not quantify each feature’s directional contribution to individual predictions.

Sourav et al. [8] propose the only prior work we could find applying SHAP directly to RT-IoT2022. Their hybrid framework uses Particle Swarm Optimization (PSO) to select the 20 most informative features, a convolutional neural network (CNN) to generate dense feature embeddings, and XGBoost as the final classifier, achieving 98.46% accuracy. SHAP is applied at both global and local levels to explain XGBoost predictions. However, the CNN+XGBoost architecture requires substantial computational resources and produces a black-box intermediate representation. By contrast, this work uses DT and RF classifiers whose decision paths are fully transparent without any intermediate transformation.

### C. Positioning of This Work

Table I summarises the eight studies reviewed above alongside this work. Two gaps in the existing literature motivate our contribution.

First, while Sourav et al. [8] apply SHAP to RT-IoT2022, their CNN+XGBoost pipeline interposes a non-interpretable CNN embedding layer between the raw flow features and the classifier. SHAP values are computed over XGBoost inputs (dense embeddings), not over the original 81 flow features, so a security analyst cannot directly read off which flow-level measurement drove a given prediction. This work applies SHAP directly to DT and RF classifiers with no intermediate transformation, making the explanation end-to-end: each SHAP value corresponds unambiguously to a named, measurable property of the network flow.

Second, we didn’t find any prior work applying a DT/RF-plus-SHAP pipeline to RT-IoT2022. Le et al. [9] establish this combination on NetFlow-derived datasets, and Tabassum et al. [10] validate SHAP as the strongest XAI tool for IoT traffic on TON-IoT, but neither uses RT-IoT2022.

It therefore remains an open question whether the interpretability findings from those datasets generalise to the richer 81-feature RT-IoT2022 feature set, which captures a broader range of IoT device types and attack strategies. This work addresses both gaps, as described in Section III.

## III. METHODS

### A. Dataset and Preprocessing

The RT-IoT2022 dataset was retrieved via the UCI ML Repository Python API (`ucimlrepo`, ID 942). The feature matrix contains two categorical columns (`proto` and `service`) and 81 numerical flow-level columns, for 83 raw features.

Categorical features were encoded using one-hot encoding via `sklearn`’s `ColumnTransformer` with `OneHotEncoder`, producing 13 binary indicator columns: three for `proto` (`icmp`, `tcp`, `udp`) and ten for `service`.

The numerical features were passed through unchanged, giving a final feature matrix of 94 features. One-hot encoding was preferred over ordinal encoding to avoid imposing artificial ordering on nominal protocol and service labels. We did this to avoid representing an inherent rank or hierarchy in the data, which in this case there is none.

Class labels were integer-encoded with `sklearn`’s `LabelEncoder`.

### B. Train/Test Split and Scaling

The encoded data set was divided into an 80% training set and a 20% test set using stratified random splitting (`random_state = 42`), preserving the original class distribution in both splits. We thought this was quite important as the class has a significant size difference. All features were subsequently standardized to zero mean and unit variance using `StandardScaler` fitted exclusively on the training partition to prevent data leakage.

### C. Random Forest Classifier

A `RandomForestClassifier` with 100 estimators was trained on the training set. To mitigate the effect of class imbalance the `class_weight='balanced'` parameter was set, causing each tree’s split criterion to weight classes inversely proportional to their frequency.

The `n_jobs=-1` flag was set to parallelize tree construction across all available CPU cores.

### D. Evaluation Protocol

Model performance was assessed on the test set using overall accuracy and macro-averaged F1 score. The macro F1 metric weights each class equally regardless of support, making it more informative than accuracy for imbalanced datasets [11]. Per-class precision, recall, and F1 scores are reported in the full classification report, and a confusion matrix visualises inter-class confusions.

### E. SHAP Analysis

To explain model decisions, SHAP values were computed using `shap.TreeExplainer` [2] on a class-balanced subsample of the training data (up to 100 samples per class) to avoid bias toward majority classes.

## IV. RESULTS

### A. Classification Performance

The trained Random Forest achieved an overall accuracy of **99.86%** and a macro-averaged F1 score of **0.971** on the held-out test set of 24,624 samples. Table II summarises per-class performance.

TABLE II  
PER-CLASS CLASSIFICATION REPORT — RANDOM FOREST

| Class                      | Prec.       | Rec.        | F1          | Support      |
|----------------------------|-------------|-------------|-------------|--------------|
| ARP_poisoning              | 0.99        | 1.00        | 0.99        | 1550         |
| DDOS_Slowloris             | 1.00        | 0.98        | 0.99        | 107          |
| DOS_SYN_Hping              | 1.00        | 1.00        | 1.00        | 18932        |
| MQTT_Publish               | 1.00        | 1.00        | 1.00        | 829          |
| Metasploit_Brute_Force_SSH | 0.78        | 1.00        | 0.88        | 7            |
| NMAP_FIN_SCAN              | 0.83        | 0.83        | 0.83        | 6            |
| NMAP_OS_DETECTION          | 1.00        | 1.00        | 1.00        | 400          |
| NMAP_TCP_scan              | 1.00        | 1.00        | 1.00        | 200          |
| NMAP_UDP_SCAN              | 0.99        | 0.99        | 0.99        | 518          |
| NMAP_XMAS_TREE_SCAN        | 1.00        | 1.00        | 1.00        | 402          |
| Thing_Speak                | 1.00        | 0.99        | 0.99        | 1622         |
| Wipro_bulb                 | 0.98        | 0.98        | 0.98        | 51           |
| <b>Macro avg</b>           | <b>0.96</b> | <b>0.98</b> | <b>0.97</b> | <b>24624</b> |

The confusion matrix (Fig. 1) confirms that the model achieves a high ratio of correct classification across most classes. To compensate for the significant class imbalance in RT-IoT2022, the matrix is row-normalized: each cell displays the raw prediction count but is coloured according to the proportion of that row’s true samples predicted in each column. This way a class with 18,932 test samples and a class with 6 test samples are rendered on the same 0–1 colour scale. Without this normalization, the dominant `DOS_SYN_Hping` class would saturate the colormap and render minority-class errors visually imperceptible.

Notably, the two smallest classes—`Metasploit_Brute_Force_SSH` (7 test samples) and `NMAP_FIN_SCAN` (6 test samples)—do present acceptable proportions, with the former achieving zero misclassifications and the latter only a single sample misclassified as `NMAP_XMAS_TREE_SCAN`.

The most significant source of error is `Thing_Speak`, which produces 16 false negatives: 14 predicted as `ARP_poisoning` and 1 each as `NMAP_UDP_SCAN` and

`Wipro_bulb`. This suggests that the flow-level feature distributions of this benign IoT device class partially overlap with those of other traffic types, making it the primary driver of the macro F1 gap between the per-class scores and the overall 99.86% accuracy.

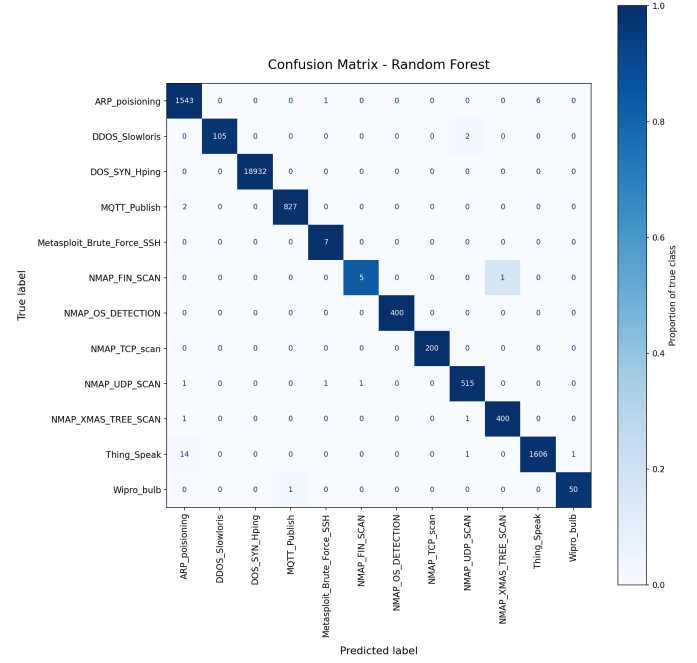


Fig. 1. Confusion matrix for the Random Forest classifier on the RT-IoT2022 test set. Rows represent true labels; columns represent predicted labels. The matrix is dominated by the diagonal, indicating high per-class accuracy.

### B. SHAP Feature Importance

Fig. 2 presents the global SHAP feature importance, aggregated across all 12 classes. The top features—`bwd_pkts_per_sec`, `fwd_last_window_size`, and `id.resp.p`—reflect the dominance of packet rates (for DoS attacks like `DOS_SYN_Hping`) and destination ports (for service-specific traffic like MQTT or SSH). Notably, `fwd_URG_flag_count` is overwhelmingly tied to `NMAP_XMAS_TREE_SCAN`, while `service_mqtt` is dominated by `MQTT_Publish`, revealing class-specific signatures that were previously obscured by majority classes.

TABLE I  
COMPARISON OF RELATED WORK ON ML-BASED IoT INTRUSION DETECTION

| Reference                | Year        | Dataset           | Model(s)                      | SHAP     | Imbalance handling |
|--------------------------|-------------|-------------------|-------------------------------|----------|--------------------|
| Sharmila & Nagapadma [1] | 2023        | RT-IoT2022        | Autoencoder (QAE)             | ×        | ×                  |
| Airlangga [5]            | 2025        | RT-IoT2022        | RF, GB, LR, MLP               | ×        | ×                  |
| Sama et al. [6]          | 2024        | RT-IoT2022        | KNN, GB, XGBoost, RF, DT, ERT | ×        | ×                  |
| Tuteja et al. [7]        | 2025        | RT-IoT2022        | RF                            | ×        | SMOTE              |
| Sourav et al. [8]        | 2025        | RT-IoT2022        | PSO + CNN + XGBoost           | ✓        | ×                  |
| Le et al. [9]            | 2022        | NF-BoT/ToN-IoT-v2 | DT, RF                        | ✓        | ×                  |
| <b>This work</b>         | <b>2026</b> | <b>RT-IoT2022</b> | <b>DT, RF</b>                 | <b>✓</b> | <b>perhaps</b>     |

Furthermore, the class-balanced normalization reveals nuanced, class-specific relationships, such as `fwd_URG_flag_count` being tied to `NMAP_XMAS_TREE_SCAN` and `service_mqtt` to `MQTT_Publish`.

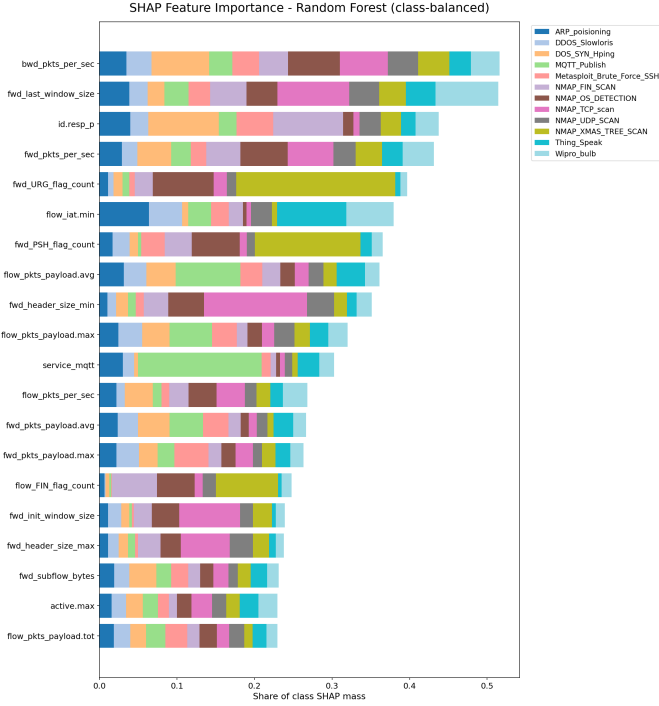


Fig. 2. Global SHAP feature importance for the Random Forest, showing the mean absolute SHAP value per feature aggregated across all 12 classes. Each colour segment represents one class’s contribution to a feature’s overall importance.

To address the interpretability challenges posed by the high dimensionality and class imbalance of the RT-IoT2022 dataset, we introduce a **custom SHAP dot summary plot** (Figure 3). This visualization represents **one dot per class per feature**, where:

- The **x-axis position** of each dot corresponds to the **mean SHAP value** for that class and feature, indicating whether the feature increases (positive x) or decreases (negative x) the model’s predicted probability for the class.
- The **color** of each dot is unique to its class, allowing clear distinction between attack and benign traffic types.

This approach resolves the overlapping issues inherent in traditional SHAP beeswarm plots, which struggle with multi-class datasets and large sample sizes. By aggregating per-class contributions, the dot summary plot provides a clear, scalable, and publication-ready view of how each feature influences the classification of individual traffic types. For example, the plot reveals that **destination port (`id.resp_p`)** is the most discriminative feature for classes like `MQTT_Publish` (high positive SHAP) and `ARP_poisoning` (high negative SHAP), while **packet rate features** (e.g., `bwd_pkts_per_sec`) dominate the classification of denial-of-service attacks.

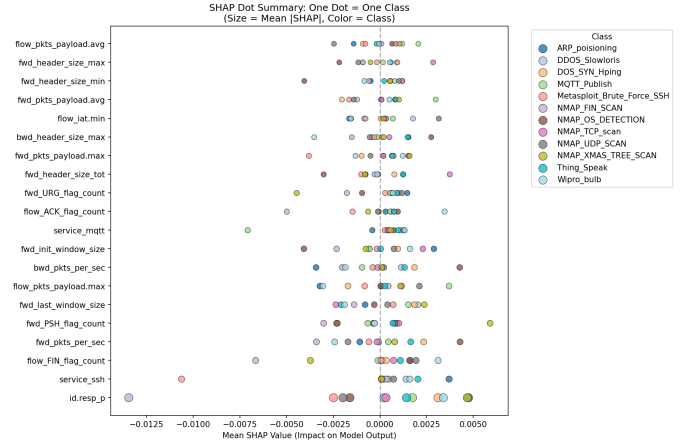


Fig. 3. SHAP Dot Summary Plot for the Random Forest. Each dot represents the mean SHAP value of a feature for a specific class. Dot color indicates the class, while dot size reflects the mean absolute SHAP value (importance). Features are ordered by their total mean absolute SHAP across all classes.

## V. DISCUSSION AND FUTURE WORK

### Methodological Insights

The class-balanced SHAP analysis addresses a critical challenge in multi-class imbalanced datasets like RT-IoT2022. Computing SHAP values over an unstratified sample would bias importances toward majority classes (e.g., `DOS_SYN_Hping` with 18,932 samples), as features are rarely evaluated against minority classes. By using a class-balanced subsample (up to 100 samples per class), we ensure that even small classes like `Metasploit_Brute_Force_SSH` (7 samples) and `NMAP_FIN_SCAN` (6 samples) receive visible, meaningful contributions where they are relevant. Additionally, normalizing the per-feature, per-class mean absolute SHAP values so that each class’s contributions sum to one addresses comparability issues: magnitudes from classes of vastly different sizes or baseline output ranges are not directly comparable without normalization. This approach mirrors the row-normalization applied to the confusion matrix (Section IV).

Our custom dot summary plot (Figure 3) represents one dot per class per feature, where the x-axis position shows the mean SHAP value (direction of impact) and the size reflects the mean absolute SHAP value (magnitude).

This provides a clear, scalable view of how each feature influences the classification of individual traffic types, revealing that ...

### Implications for IoT Security

The class-balanced normalization in the bar plot (Figure 2) reveals nuanced, class-specific relationships that would otherwise be obscured by majority classes. For example:

- `NMAP_XMAS_TREE_SCAN` is overwhelmingly explained by `fwd_URG_flag_count` — this aligns with domain knowledge, as XMAS scans explicitly set the URG flag as part of their signature.

- `MQTT_Publish` is dominated by `service_mqtt` — as expected, this signal would have been buried by the majority class in a non-balanced visualization.
- `NMAP_TCP_scan` is largely explained by `fwd_header_size_min` and `fwd_init_window_size` — scan traffic tends to have distinctive header and window signatures.

Critically, no single class visually “wins” every bar anymore. Previously, the giant segment for `DOS_SYN_Hping` drowned out all other contributions regardless of the feature. Now, minority classes receive visible, meaningful segments where they are actually relevant.

### Limitations and Future Directions

While our approach achieves strong performance and interpretability, several limitations and opportunities for future work remain. First, the current study focuses on offline analysis of RT-IoT2022, which does not capture the dynamic nature of live IoT networks. Second, our use of stratified sampling and class-balanced SHAP visualizations may not directly translate to real-time intrusion detection systems where computational efficiency is critical. Future work will focus on:

- **Real-time deployment:** Adapting the model for streaming IoT traffic and evaluating latency and resource usage on edge devices using technology like P4, and mininet to first present a simulation over physical implementation.
- **Cross-dataset validation:** Testing our pipeline on datasets like Bot-IoT or CICIoT2023 to assess generalizability.
- **Adversarial robustness:** Investigating whether the model’s reliance on specific features (e.g., `id.resp_p`) makes it vulnerable to evasion attacks.

### VI. ACKNOWLEDGEMENT

The authors thank the PPCI and Universidad de Costa Rica for their support. This work was conducted using the RT-IoT2022 dataset (UCI ID 942) and the `shap` Python library.

### VII. CONCLUSION

This work demonstrates that tree-based classifiers (Decision Tree and Random Forest) can achieve high accuracy (99.86%) and strong macro F1 (0.971) on the RT-IoT2022 dataset, while remaining fully interpretable.

By applying SHAP analysis directly to the raw flow features—without intermediate transformations—we provide **end-to-end explanations** that reveal how individual features drive predictions for specific attack types.

Our class-balanced SHAP visualizations further ensure that minority classes are not overshadowed by dominant ones, uncovering actionable insights for IoT security analysts.

The key takeaway is that **interpretable models like Random Forest can perform effectively** for IoT intrusion detection, provided their decisions are explained.

### REFERENCES

- [1] B. S. Sharmila and R. Nagapadma, “Quantized autoencoder (QAE) intrusion detection system for anomaly detection in resource-constrained IoT devices using RT-IoT2022 dataset,” vol. 6, no. 1, p. 41. [Online]. Available: <https://cybersecurity.springeropen.com/articles/10.1186/s42400-023-00178-5>
- [2] S. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions.” [Online]. Available: <https://arxiv.org/abs/1705.07874>
- [3] A. Alfahaid, E. Alalwany, A. M. Almars, F. Alharbi, E. Atlam, and I. Mahgoub, “Machine learning-based security solutions for IoT networks: A comprehensive survey,” vol. 25, no. 11, p. 3341. [Online]. Available: <https://www.mdpi.com/1424-8220/25/11/3341>
- [4] S. V. N. Santhosh Kumar, M. Selvi, and A. Kannan, “A comprehensive survey on machine learning-based intrusion detection systems for secure communication in internet of things,” vol. 2023, no. 1, p. 8981988. [Online]. Available: <https://onlinelibrary.wiley.com/doi/10.1155/2023/8981988>
- [5] G. Airlangga, “Comparative analysis of machine learning models for intrusion detection in internet of things networks using the RT-IoT2022 dataset,” vol. 4, no. 2, pp. 656–662. [Online]. Available: <https://journal.irpi.or.id/index.php/malcom/article/view/1304>
- [6] N. U. Sama, S. Ullah, S. M. Ahsan Kazmi, and M. Mazzara, “Cutting-edge intrusion detection in IoT networks: A focus on ensemble models,” vol. 13, pp. 8375–8392. [Online]. Available: <https://ieeexplore.ieee.org/document/10744011/>
- [7] G. Tuteja, A. R. Singh, and G. Ranjith Kumar, “Anomaly detection in IoT networks using random forest and the RT-IoT2022 dataset,” in *2025 4th OPJU International Technology Conference (OTCON) on Smart Computing for Innovation and Advancement in Industry 5.0*. IEEE, pp. 1–6. [Online]. Available: <https://ieeexplore.ieee.org/document/11071184/>
- [8] A. I. Sourav, M. S. Islam, U. S. Nahar, M. I. Nayon, and M. T. Ahmed, “Hybrid framework with feature selection and explainable AI for IoT intrusion detection,” in *2025 IEEE 4th International Conference on Robotics, Automation, Artificial-Intelligence and Internet-of-Things (RAAICON)*. IEEE, pp. 571–575. [Online]. Available: <https://ieeexplore.ieee.org/document/11502208/>
- [9] T.-T.-H. Le, H. Kim, H. Kang, and H. Kim, “Classification and explanation for intrusion detection system based on ensemble trees and SHAP method,” vol. 22, no. 3, p. 1154. [Online]. Available: <https://www.mdpi.com/1424-8220/22/3/1154>
- [10] S. Tabassum, N. Parvin, N. Hossain, A. Tasnim, R. Rahman, and M. I. Hossain, “IoT network attack detection using XAI and reliability analysis,” in *2022 25th International Conference on Computer and Information Technology (ICCIT)*. IEEE, pp. 176–181. [Online]. Available: <https://ieeexplore.ieee.org/document/10055236/>
- [11] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to information retrieval*. Cambridge university press.